第二次基础实验 实验文档

Last Upd: 2026.10.03

- 第二次基础实验 实验文档
  - 01 实验目标与任务
    - 1 容器内遍历
    - 2 嵌套循环与推导式
    - 3 二分查找
    - 4 图像卷积
  - 02 验收内容

01 实验目标与任务

1 容器内遍历

目标是考察分支检查、 `while` 循环与 `for in` 语句的容器内遍历操作。

编写程序实现基于偏移量的字符串加密。加密的过程是对原字符串中的每个字符对应的 Unicode 值加上一个偏移量，然后将得到的 Unicode 值映射到该字符对应的加密字符。

用户输入一个不小于-15的非零整数和一个由大小写字母或数字组成的字符串，程序生成并输出加密得到的字符串。例如用户输入 10 和字符串 “Attackat1600” 得到的加密字符串是 “K~~kmuk~;@::”。

要求程序的工作流如下：

1. 使用 `while` 循环重复接受用户的输入（通过 `input()` 函数），并用 `if` 语句检查输入合法性。若输入不合法（可以使用 `type()` 或 `isinstance()` 检查变量类型），则提示并要求用户重新输入直到输入内容合法。
2. 得到合法输入之后，利用 `for in` 语句遍历字符串，以对其中每个字符执行移位操作。移位操作可通过修改其 ASCII/Unicode 码实现，需要调用函数 `chr()` 和 `ord()` 。
3. 处理完成后，输出原字符串和加密结果。

2 嵌套循环与推导式

目标是考察推导式初始化容器、 `random` 库以及嵌套 `for` 循环。

编写程序实现随机向量内积和矩阵乘法。要求用户输入向量维数 d、矩阵行数 m 与列数 n，实现用推导式随机初始化两个长度为 d 的整数元素向量 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$，以及两个整数元素矩阵 $A, B \in \mathbb{R}^{m \times n}$，并求解向量内积和矩阵乘法。

要求程序的工作流如下：

1. 使用 `while` 循环重复接受用户的输入（通过 `input()` 函数），并用 `if` 语句检查输入合法性（维数应当为正整数）。若输入不合法，则提示并要求用户重新输入直到输入内容合法。
2. 得到合法输入之后，利用推导式语句和 `random` 库中的 `randint()` 函数，生成元素取值为 $[0, 10]$ 的 d 维向量（用一维列表表示）和 $m \times n$ 矩阵（用二维列表表示）。**注意，本实验不允许使用 `numpy` 、 `scipy` 等计算库！** 验收的时候别给逮到了。
3. 利用 `for in` 循环计算向量内积；利用嵌套 `for in` 循环计算矩阵乘法（这里假设矩阵乘法为 $A^T B$，即输出为 $n \times n$ 的矩阵）。
4. 输出随机生成的向量和矩阵，并输出运算结果。

一个说明：通过这种顺序执行的方式计算向量内积和矩阵乘法是十分低效的，自己测试的时候尽量别填太大的维数 qwq。也正因为如此，利用 CPU 和 GPU 的并行计算特性加速它们的计算才显得尤为重要，这也是当今模型对 GPU 算力的需求如此之大的主要原因之一。

3 二分查找

编写一个程序使用二分法查找给定的包含若干整数的列表 `s` 中是否存在给定的整数 `k` 。使用二分查找的前提是列表已按照从小到大的顺序排序。为此，程序需要先判断 `s` 是否已经排序。若未排序，则需调用 `qsort` 函数进行排序并输出排序结果。

示例代码已经给出，需要实现函数 `is_sorted` 和递归函数 `binary_search` 。 `binary_search` 在列表 `s` 的索引值属于闭区间 `[low,high]` 的元素中查找 `k` ，若找到则返回 `k` 的索引值，否则返回 -1。

```python
# binary_search.py

def is_sorted(s):

    ### TODO 1: 补全判断是否排序 ###

    ### TODO 1 结束 ###

def qsort(s):
    if len(s) <= 1: return s
    s_less = []; s_greater = []; s_equal = []
    for k in s:
        if k < s[0]:
            s_less.append(k)
        elif k > s[0]:
            s_greater.append(k)
        else:
            s_equal.append(k)
    return qsort(s_less) + s_equal + qsort(s_greater)

def binary_search(s, low, high, k):

    ### TODO 2: 补全递归二分查找 ###

    ### TODO 2 结束 ###

    s = [5, 6, 21, 32, 51, 60, 67, 73, 77, 99]
    if not is_sorted(s):
        s = qsort(s)
        print(s)

print(binary_search(s, 0, len(s) - 1, 5))
print(binary_search(s, 0, len(s) - 1, 31))
print(binary_search(s, 0, len(s) - 1, 99))
print(binary_search(s, 0, len(s) - 1, 64))
print(binary_search(s, 0, len(s) - 1, 51))
```

4 图像卷积

目标是考察嵌套 `for in` 循环。

卷积神经网络中的卷积层涉及到图像卷积计算，其来源于数字图像处理领域。一个卷积核实际上可以视为一个滤波器，它在图像上逐位置滑动，在每个位置取出一个与卷积核大小相同的局部区域，将局部区域中的元素与卷积核对应位置的元素相乘并求和，由此得到输出特征图中的一个元素。

一张数字灰度图像可以看作一个矩阵，矩阵中的每个元素表示对应像素的灰度值，通常取值范围为 $0 \sim 255$。对于彩色图像，其矩阵 X 的每个元素 $\mathbf{x}_{ij}$ 都是一个向量（即此时图像表示为一个张量），其记录了 (i, j) 位置像素的 RGB 值。

给定一个卷积核，例如如下的 3×3 卷积核，它实现了对图片执行均值模糊的操作：

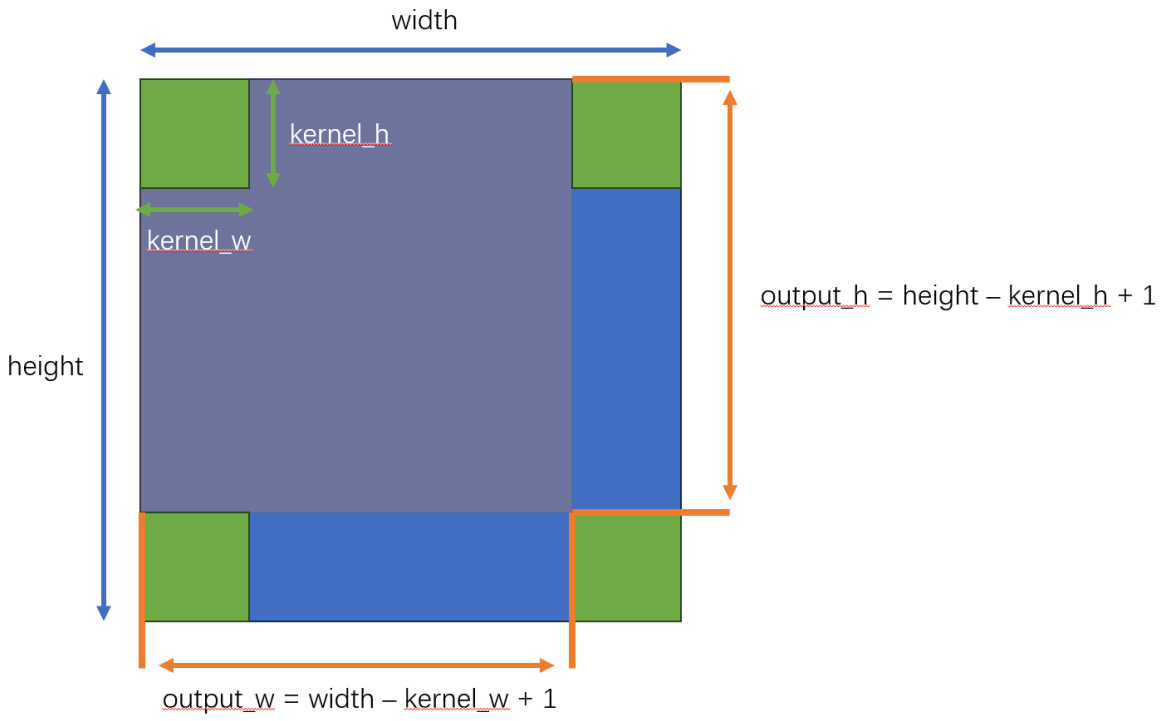
$$K_{blur} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

和一个 256×256 的灰度图像

$$X \in \mathbb{R}^{256 \times 256}, \qquad x_{ij} \equiv \{X\}_{ij} \in [0, 255]$$

一个一般的图像卷积过程可以如下描述：

1. 计算卷积核遍历的实际高度和宽度 `output_h` 和 `output_w`。因为卷积核存在大小，若以其左上角元素为坐标进行滑动，它不可能遍历原图像的每个像素。



2. 当卷积核左上角元素移动至 (i, j) 位置时，其输出的结果即为输出图 (i, j) 位置的像素灰度值 y_{ij}。对于每一个位置，将卷积核中的元素与图像中对应位置的像素值分别相乘，再把所有乘积相加，

即可得到输出图像中的一个像素。对于 3×3 的卷积核，其输出像素灰度值为

$$\begin{aligned} y_{i,j} = & x_{i,j}K_{0,0} + x_{i,j+1}K_{0,1} + x_{i,j+2}K_{0,2} \\ & + x_{i+1,j}K_{1,0} + \cdots \\ & + x_{i+2,j+2}K_{2,2}. \end{aligned}$$

一般地，

$$y_{i,j} = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} image_{i+u,j+v}\, kernel_{u,v}$$

其中，K_h 和 K_w 分别对应卷积核的高（kernel height）和宽（kernel width）。

因此，实现二维卷积需要同时遍历：

```
输出图像的行（output_h）
    输出图像的列（output_w）
        卷积核的行（kernel_h）
            卷积核的列（kernel_w）
```

这是一个典型的四层嵌套 `for in` 循环。（行和列的位置实际可以互换）

本实验要求补全附件 `image_convolution_todo.py` 中这部分嵌套循环的代码（已标记为 TODO 部分），以实现对单张灰度图像执行四个不同的操作（对应四种不同的卷积核）：均值模糊、高斯模糊、图像锐化、边缘检测。

代码补全后，可通过 `uv` 创建环境并添加相应的库并执行代码

```
uv python pin 3.12 # 固定当前环境的 python 版本
uv init # 创建虚拟环境
uv sync # 同步虚拟环境
uv add opencv-python numpy matplotlib # 添加所需的库
uv run python image_convolution_todo.py
```

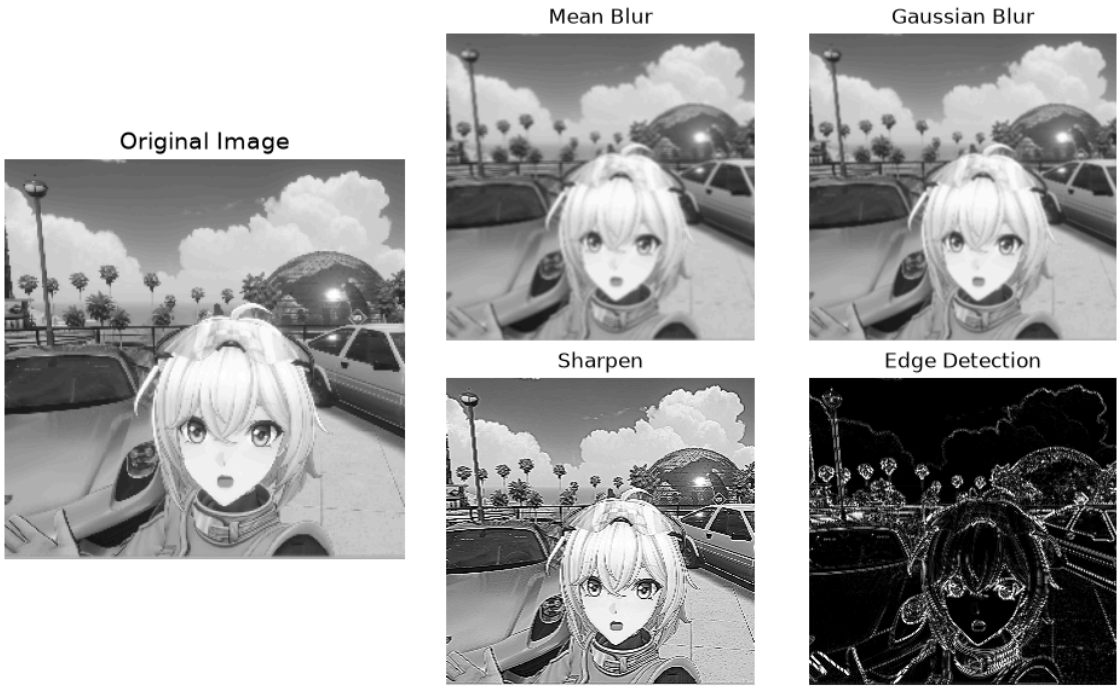
一些补充说明：

1. 程序会自动读取同目录下的 `image.jpg` 图像，自动将其压缩为 256×256 大小，并转化为灰度图。你可以自行提供输入图片（图片比例应为 1:1），也可以直接使用实验附件中的示例图。
2. 本实验使用的库分别为 `opencv` 、 `numpy` 和 `matplotlib` 。 `opencv` 是一个经典的图像处理库， `numpy` 为向量计算库，对本实验的任务而言，其仅提供了对向量的表示，并不刚需其它任何计

算。 `matplotlib` 是一个经典的图像绘制与显示库。**注意，本实验不得直接调用这些库中的计算方法！**

3. 本实验中的图像卷积实际是沿用了深度学习中的计算约定，即不对卷积核进行翻转。数学上严格定义的卷积是需要对计算核进行翻转的。

一个示例输出如下：



02 验收内容

实验 1 要求能够检测异常输入，且能够正确运行提供的示例。

实验 2 要求能够正确运行且输出结果，在低维情况下可手动计算验证。

实验 3 要求输出结果正确。

实验 4 要求正确填补代码，且有正确的图像输出结果。